Single-Cycle RISC-V Processor Verification Report

R, I, S, B, and J Type Instruction Set Architecture

Muhammad Tahir Zia
(25ins-01005)

Submitted to Sir Abbass

Contents

List of Figures

List of Tables

1 Introduction

This report documents the verification of a 32-bit, single-cycle RISC-V processor developed in Verilog [3]. The design supports a subset of the RISC-V base instruction set [1], specifically encompassing:

- **R-Type** (Register-to-register arithmetic)
- **I-Type** (Immediate arithmetic and loads)
- **S-Type** (Store operations)
- **B-Type** (Conditional branches)
- **J-Type** (Unconditional jumps)

Verification was performed using individual unit-level testbenches for every hardware module, followed by an integrated top-level simulation that executed a specific assembly program loaded into the instruction memory. The methodology follows standard verification practices described in [2, 4, 5].

2 Architectural Overview

The processor adopts a classic single-cycle microarchitecture where all instructions execute within one clock cycle [2, 6]. The data path is composed of the following standard components:

- **Program Counter (PC):** (`program_counter.sv`) 32-bit synchronous counter with a synchronous reset.
- **Instruction Memory:** Byte-addressable read-only memory initialized with a custom test program (`inst_mem.sv`).
- **Register File:** A 32×32-bit asynchronous-read, synchronous-write register file with 5-bit read/write addresses, supporting a hard-wired $x0$ register (`reg_file.sv`).
- **Immediate Generator:** Extracts sign-extended immediate values for I, S, B, and J type instructions based on the `ImmSrc` control signal (`imm_gen.sv`).
- **Arithmetic Logic Unit (ALU):** A combinational block implementing addition, subtraction, bitwise AND, OR, and set-less-than (SLT) operations (`alu_logic.sv`).
- **Data Memory:** (`data_mem.sv`) Synchronous write, asynchronous read byte-addressable memory.
- **Control Unit:** A hierarchical decoder consisting of a `main_decoder` and an `alu_decoder`.
- **Adders and Multiplexers:** A 32-bit adder for PC+4 and branch target calculation, along with 2-to-1 multiplexers.

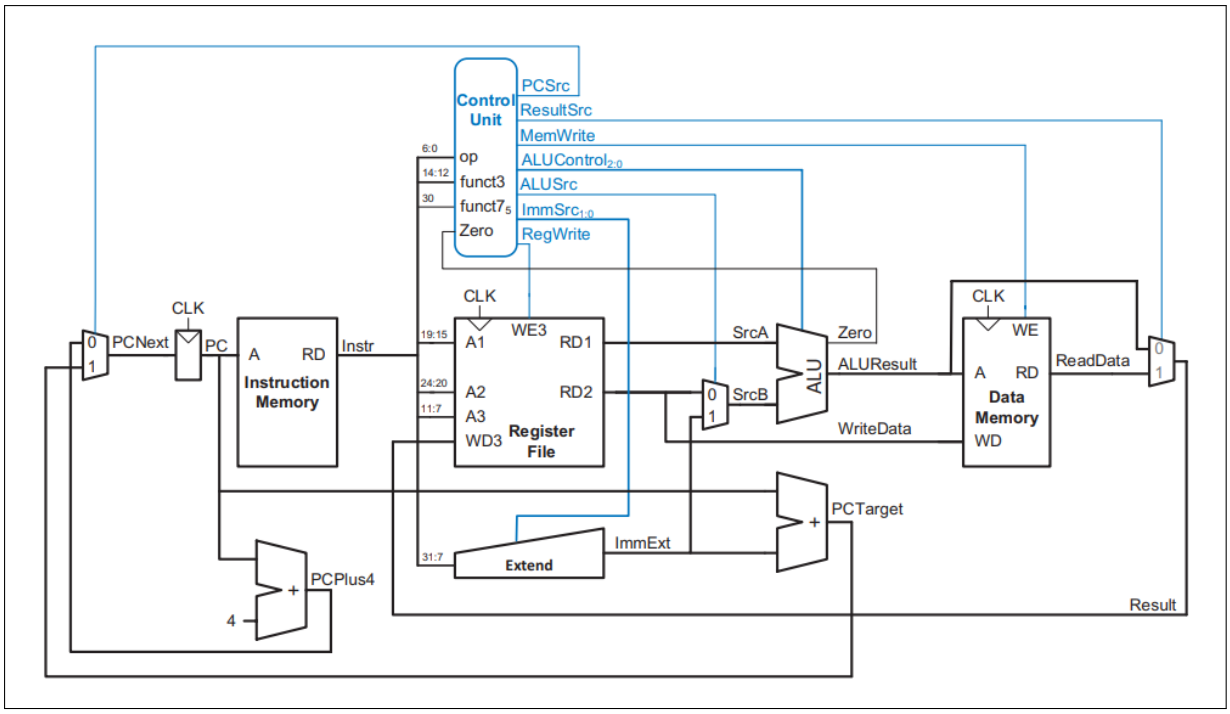

Figure 1: Block diagram of the single-cycle RISC-V processor datapath, based on the classic single-cycle architecture [2].

3 Instructions Under Test

The test program uses the following instructions, whose machine encoding fields are summarized in Table 1. These encodings follow the official RISC-V instruction set manual [1].

Table 1: Instructions and their RISC-V encoding fields

Instruction	Opcode	funct3	funct7	rd	rs1	rs2	imm	Machine Code
JAL x3, 12	1101111	—	—	00011	—	—	0x00C	0x00C001EF
ADD x2, x1, x1	0110011	000	0000000	00010	00001	00001	—	0x00108133
SW x2, 4(x0)	0100011	010	—	—	00000	00010	0x004	0x00202223
LW x1, 0(x0)	0000011	010	—	00001	00000	—	0x000	0x00002083
BEQ x0, x0, -12	1100011	000	—	—	00000	00000	0xFF4	0xFE000AE3

The immediate values are sign-extended according to their format: J-type (20-bit), B-type (13-bit including implied zero), S-type (12-bit), and I-type (12-bit).

4 Module-Level Verification

Each core module was independently verified using dedicated Verilog testbenches to ensure correct combinatorial and sequential behavior.

4.1 Program Counter and Adders

The PC increments by 4 on every rising edge of the clock following a reset. The 32-bit adder module correctly computed the sums of various test vectors provided by the testbench. This

straightforward verification mirrors standard PC testing approaches [2, 8].

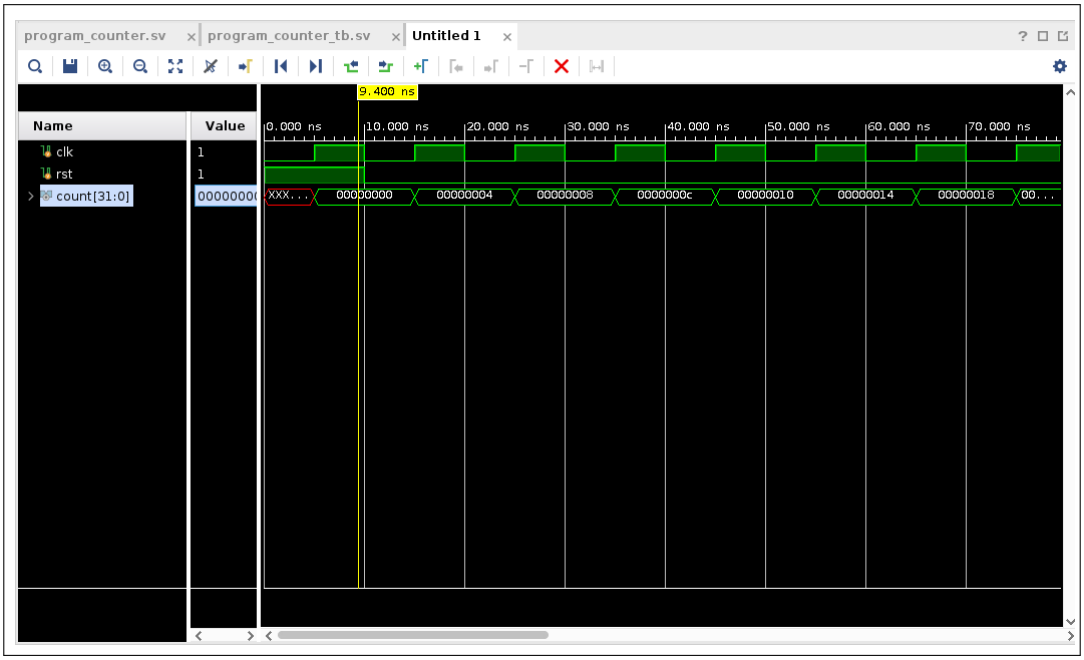

Figure 2: Program Counter simulation waveform showing reset and sequential increment.

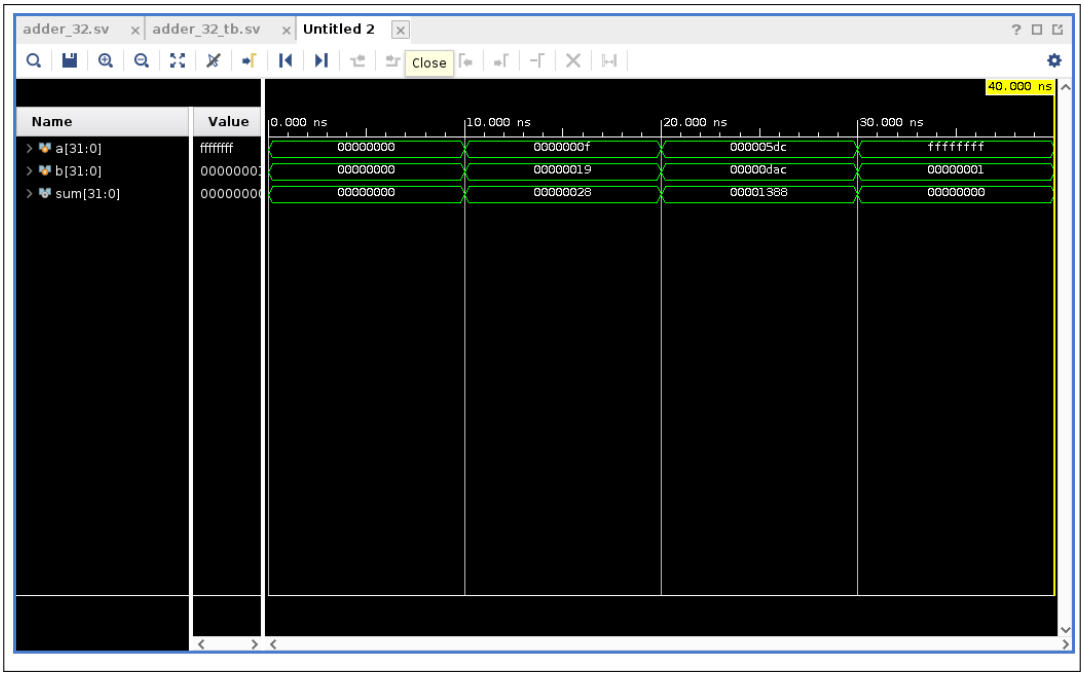

Figure 3: 32-bit Adder testbench verifying arithmetic sums.

4.2 ALU Operations

The ALU was tested for its various arithmetic and logical instructions. The `alu_logic.sv` module implements standard operations (ADD, SUB, AND, OR, SLT) based on the 3-bit `alu_control` signal. The testbench verified operation correctness and the `zero_flag` assertion [2, 9].

Table 2: ALU Operations Mapping

alu_control	Operation
3'b000	ADD
3'b001	SUB
3'b010	AND
3'b011	OR
3'b101	SLT

Verilog code of the ALU module

```
module ALU(
    input       [31:0] in1,
    input       [31:0] in2,
    input       [2:0]  alu_control,
    output reg  [31:0] result,
    output reg         zero_flag
);
    always @(*) begin
        result = 32'b0;
        case(alu_control)
            3'b000: result = in1 + in2;
            3'b001: result = in1 - in2;
            3'b010: result = in1 & in2;
            3'b011: result = in1 | in2;
            3'b101: result = ($signed(in1) < $signed(in2)) ? 32'd1 : 32'
    d0;
            default: result = 32'b0;
        endcase
        zero_flag = (result == 32'b0) ? 1'b1 : 1'b0;
    end
endmodule
```

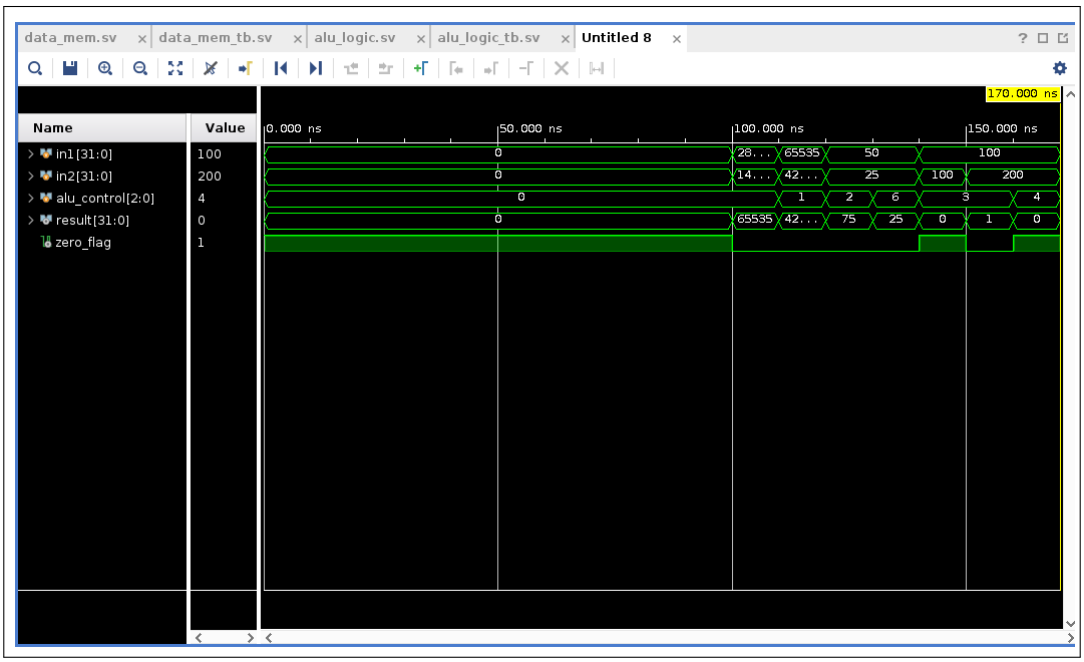

Figure 4: ALU logic simulation demonstrating various operations and the zero flag.

4.3 Control Unit

The control unit was verified against all supported instruction opcodes. The `main_decoder` and `alu_decoder` correctly map instruction fields to data path control signals (e.g., `ALUSrc`, `RegWrite`, `Branch`) and ALU operations. Here at 5000 ns we can see that in the testbench, when the opcode is 3 it signals that it is an I-type instruction only for (lb, lh, lw, lbu, lhu) instructions. The RegWrite signal is 1 and ImmSrc [1:0] is 0. ALUSrc is 1 and MemWrite is 0. ResultSrc is 0 and Branch is also 0. These are the control signals that go to some of the hardware to choose what to perform.

Top-level control unit integration

```
module control_unit(
    input  [6:0] op,
    input  [2:0] funct3,
    input        funct7_5,
    input        Zero,
    output       PCSrc,
    output [1:0] ResultSrc,
    output       MemWrite,
    output       ALUSrc,
    output [1:0] ImmSrc,
    output       RegWrite,
    output [2:0] ALUControl
);
    wire [1:0] ALUOp;
    wire       Branch;
    wire       Jump;

    assign PCSrc = Jump | (Branch & Zero);

    main_decoder md_inst (
        .Op(op),
        .RegWrite(RegWrite),
        .ImmSrc(ImmSrc),
        .ALUSrc(ALUSrc),
        .MemWrite(MemWrite),
        .ResultSrc(ResultSrc),
        .Branch(Branch),
        .ALUOp(ALUOp),
        .Jump(Jump)
    );

    alu_decoder ad_inst (
        .ALUOp(ALUOp),
        .funct3(funct3),
        .op_5(op[5]),
        .funct7_5(funct7_5),
        .ALUControl(ALUControl)
    );
endmodule
```

Main decoder generating control signals

```
module main_decoder(
    input       [6:0] Op,
    output reg        RegWrite,
    output reg [1:0]  ImmSrc,
    output reg        ALUSrc,
    output reg        MemWrite,
    output reg [1:0]  ResultSrc,
    output reg        Branch,
    output reg [1:0]  ALUOp,
    output reg        Jump
);
    always @(*) begin
        case(Op)
            // lw instruction
            7'b0000011: begin
                RegWrite  = 1'b1;  ImmSrc = 2'b00; ALUSrc = 1'b1;
                MemWrite  = 1'b0;  ResultSrc = 2'b01; Branch = 1'b0;
                ALUOp     = 2'b00; Jump = 1'b0;
            end
            // sw instruction
            7'b0100011: begin
                RegWrite  = 1'b0;  ImmSrc = 2'b01; ALUSrc = 1'b1;
                MemWrite  = 1'b1;  ResultSrc = 2'bxx; Branch = 1'b0;
                ALUOp     = 2'b00; Jump = 1'b0;
            end
            // R-type
            7'b0110011: begin
                RegWrite  = 1'b1;  ImmSrc = 2'bxx; ALUSrc = 1'b0;
                MemWrite  = 1'b0;  ResultSrc = 2'b00; Branch = 1'b0;
                ALUOp     = 2'b10; Jump = 1'b0;
            end
            // beq
            7'b1100011: begin
                RegWrite  = 1'b0;  ImmSrc = 2'b10; ALUSrc = 1'b0;
                MemWrite  = 1'b0;  ResultSrc = 2'bxx; Branch = 1'b1;
                ALUOp     = 2'b01; Jump = 1'b0;
            end
            // jal
            7'b1101111: begin
                RegWrite  = 1'b1;  ImmSrc = 2'b11; ALUSrc = 1'bx;
                MemWrite  = 1'b0;  ResultSrc = 2'b10; Branch = 1'b0;
                ALUOp     = 2'bxx; Jump = 1'b1;
            end
            default: begin
                RegWrite = 1'b0;  ImmSrc = 2'b00; ALUSrc = 1'b0;
                MemWrite = 1'b0;  ResultSrc = 2'b00; Branch = 1'b0;
                ALUOp = 2'b00;    Jump = 1'b0;
            end
        endcase
    end
endmodule
```

ALU decoder generating ALU control signals

```
module alu_decoder(
    input       [1:0] ALUOp,
    input       [2:0] funct3,
    input             op_5,
    input             funct7_5,
    output reg  [2:0] ALUControl
);
    always @(*) begin
        case (ALUOp)
            2'b00: ALUControl = 3'b000;
            2'b01: ALUControl = 3'b001;
            2'b10: begin
                case (funct3)
                    3'b000: begin
                        if (op_5 & funct7_5)
                            ALUControl = 3'b001;
                        else
                            ALUControl = 3'b000;
                    end
                    3'b010: ALUControl = 3'b101;
                    3'b110: ALUControl = 3'b011;
                    3'b111: ALUControl = 3'b010;
                    default: ALUControl = 3'b000;
                endcase
            end
            default: ALUControl = 3'b000;
        endcase
    end
endmodule
```

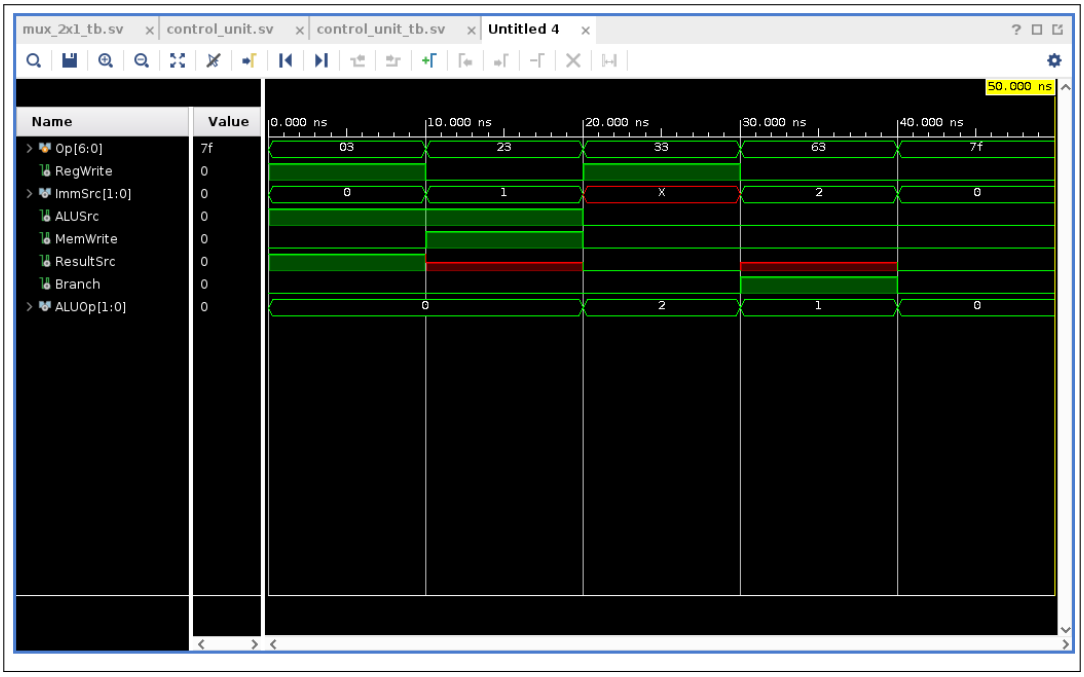

Figure 5: Control Unit simulation waveforms showing correct signal assertion for different op-codes.

4.4 Instruction Decoder and Immediate Generator

The instruction decoder splits the instruction field correctly to drive the register file and multiplexers (Image 5). The `imm_gen` module correctly handles sign extension across all formats, generating expected 32-bit immediates for I, S, B, and J instructions. For example, at **5000 ns** the instruction code to be decoded is **32'h00_31_00_B3**. According to this opcode **(0110011)** it is an R-type instruction. Therefore, after attaching all the corresponding fields the instruction becomes **add x2, x1, x1**.

Instruction field decoder

```
module decoder(
    input       [31:0] instruction,
    output reg  [2:0]  func3,
    output reg  [6:0]  func7,
    output reg  [4:0]  rs1, rs2, rd,
    output reg  [11:0] imm
);
    parameter r_type = 7'b0110011,
              s_type = 7'b0100011,
              I_type = 7'b0000011,
              b_type = 7'b1100011,
              j_type = 7'b1101111;

    wire [6:0] opcode = instruction[6:0];
    always@(*) begin
        case(opcode)
            r_type: begin
                rd = instruction[11:7]; func3 = instruction[14:12];
                rs1 = instruction[19:15]; rs2 = instruction[24:20];
                func7 = instruction[31:25]; imm = 12'bx;
            end
            s_type: begin
                imm = {instruction[31:25], instruction[11:7]};
                func3 = instruction[14:12]; rs1 = instruction[19:15];
                rs2 = instruction[24:20]; rd = 5'bx; func7 = 7'bx;
            end
            I_type: begin
                rd = instruction[11:7]; func3 = instruction[14:12];
                rs1 = instruction[19:15]; imm = instruction[31:20];
                func7 = 7'bx; rs2 = 5'bx;
            end
            b_type: begin
                func3 = instruction[14:12]; rs1 = instruction[19:15];
                rs2 = instruction[24:20];
                imm = {instruction[31], instruction[30:25], instruction
    [11:8], instruction[7]};
                func7 = 7'bx; rd = 5'bx;
            end
        endcase
    end
endmodule
```

Immediate generation with sign extension for all formats

```
module imm_Gen(
    input       [31:0] instruction,
    input       [1:0]  ImmSrc,
    output reg  [31:0] immediate_output
);
    wire [11:0] load_im   = instruction[31:20];
    wire [11:0] store_im  = {instruction[31:25], instruction[11:7]};
    wire [12:0] branch_im = {instruction[31], instruction[7], instruction
    [30:25], instruction[11:8], 1'b0};
    wire [20:0] jump_im   = {instruction[31], instruction[19:12],
    instruction[20], instruction[30:21], 1'b0};

    always @(*) begin
        case(ImmSrc)
            2'b00: immediate_output = {{20{load_im[11]}}, load_im};
            2'b01: immediate_output = {{20{store_im[11]}}, store_im};
            2'b10: immediate_output = {{19{branch_im[12]}}, branch_im};
            2'b11: immediate_output = {{11{jump_im[20]}}, jump_im};
            default: immediate_output = 32'b0;
        endcase
    end
endmodule
```

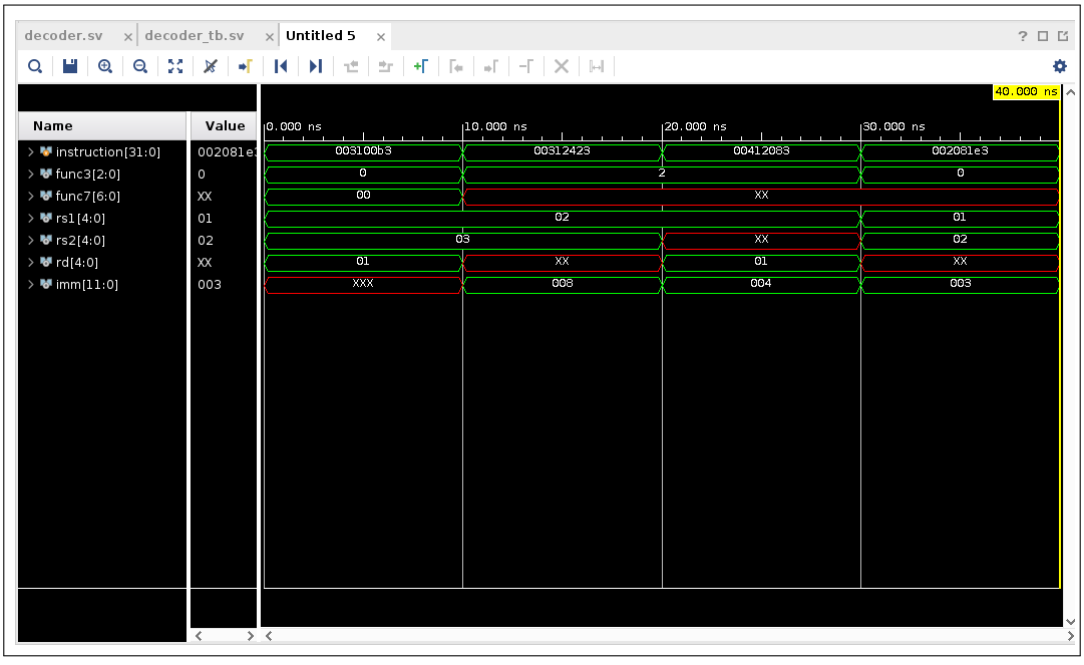

Figure 6: Instruction Decoder splitting RISC-V instruction fields.

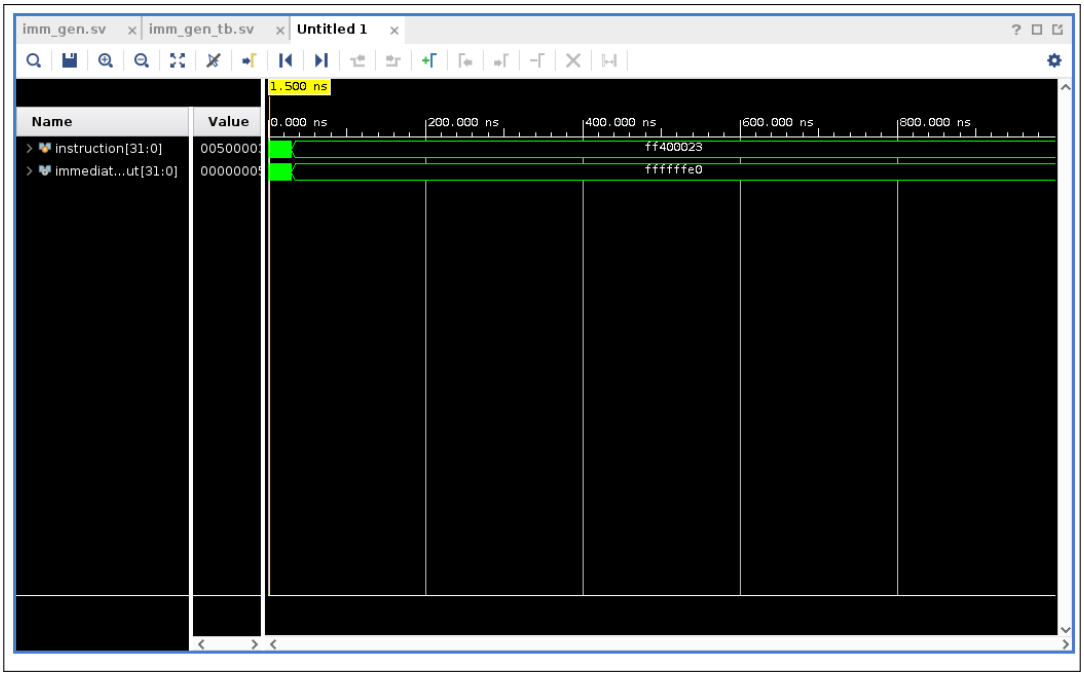

Figure 7: Immediate Generation module verifying sign extension.

4.5 Register File and Multiplexers

The register file passed all checks; writes are synchronous when `regwrite` is high, and writing to the hard-wired $x0$ is correctly ignored. The 2-to-1 and 3-to-1 multiplexers correctly pass data based on selection bits. For example, the value 99 is not being stored in $x0$ because it is hardwired zero.

Register file with synchronous write and hardwired x0

```
module REG_FILE (
    input       [4:0]  read_reg_num1, read_reg_num2, write_reg,
    input       [31:0] write_data,
    output      [31:0] read_data1, read_data2,
    input              regwrite, clock, reset
);
    reg [31:0] reg_memory [31:0];
    integer i;

    always@(posedge clock or posedge reset) begin
        if (reset) begin
            for (i = 0; i < 32; i = i + 1) begin
                reg_memory[i] <= 32'b0;
            end
        end
        else if (regwrite && (write_reg != 0)) begin
            reg_memory[write_reg] <= write_data;
        end
    end

    assign read_data1 = reg_memory[read_reg_num1];
    assign read_data2 = reg_memory[read_reg_num2];
endmodule
```

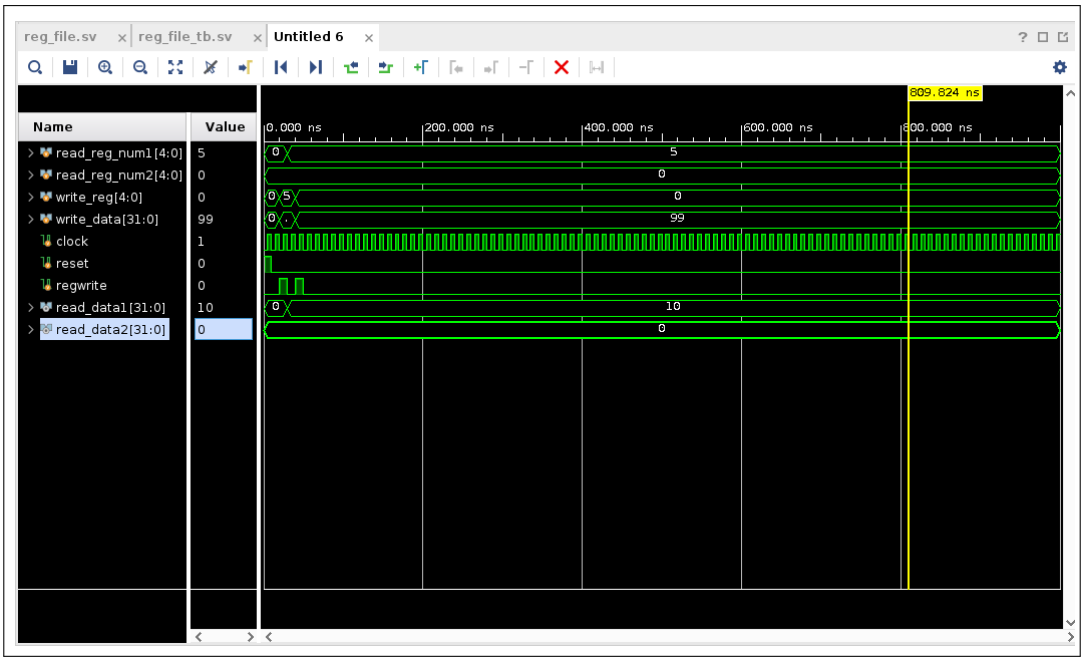

Figure 8: Register File simulation showing reads and locked x0 write.

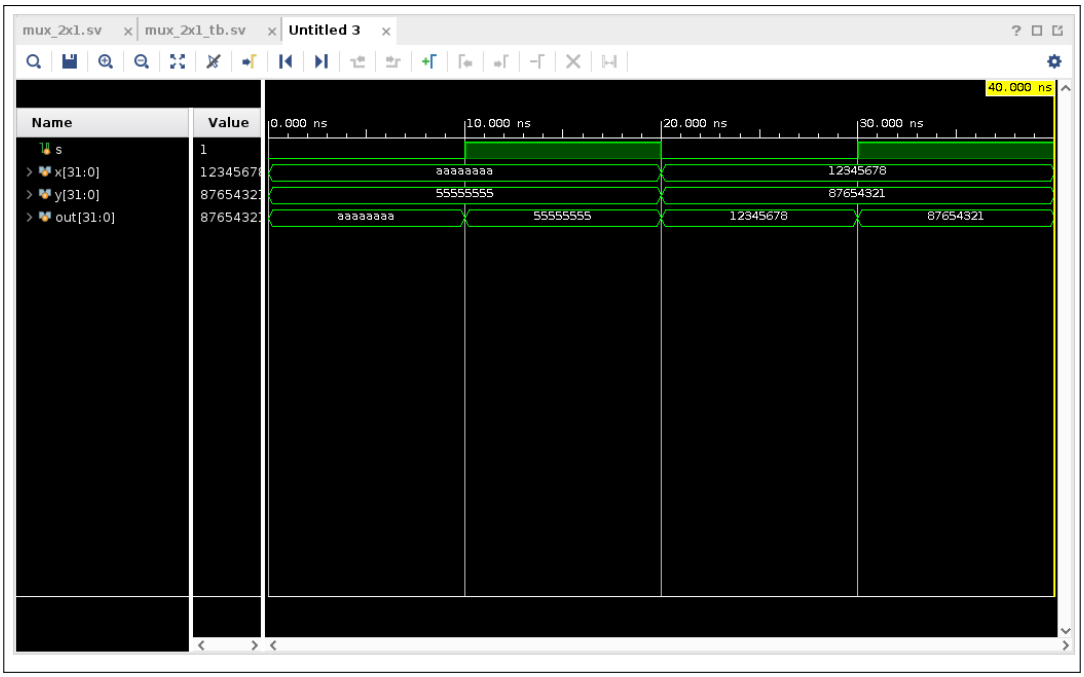

Figure 9: 2-to-1 Multiplexer verifying data routing.

4.6 Instruction and Data Memories

The instruction memory successfully outputs the 32-bit instruction code based on the PC input. The data memory was verified for synchronous writes and reads across multiple cycles, following the behavioral model described in [2].

Instruction memory initialisation and read logic

```
module INST_MEM(
    input  [31:0] PC,
    output [31:0] Instruction_Code
);
    reg [7:0] Memory [31:0];

    initial begin
        // Instruction 0: J-Type (jal x3, 12) -> 00_C0_01_EF
        Memory[0] = 8'hEF; Memory[1] = 8'h01;
        Memory[2] = 8'hC0; Memory[3] = 8'h00;
        // Instruction 1: R-Type (add x2, x1, x1) -> 00_10_81_33
        Memory[4] = 8'h33; Memory[5] = 8'h81;
        Memory[6] = 8'h10; Memory[7] = 8'h00;
        // Instruction 2: S-Type (sw x2, 4(x0)) -> 00_20_22_23
        Memory[8]  = 8'h23; Memory[9]  = 8'h22;
        Memory[10] = 8'h20; Memory[11] = 8'h00;
        // Instruction 3: I-Type (lw x1, 0(x0)) -> 00_00_20_83
        Memory[12] = 8'h83; Memory[13] = 8'h20;
        Memory[14] = 8'h00; Memory[15] = 8'h00;
        // Instruction 4: B-Type (beq x0, x0, -12) -> FE_00_0A_E3
        Memory[16] = 8'hE3; Memory[17] = 8'h0A;
        Memory[18] = 8'h00; Memory[19] = 8'hFE;
    end

    assign Instruction_Code = {Memory[PC+3], Memory[PC+2], Memory[PC+1],
    Memory[PC]};
endmodule
```

Data memory with byte-addressable synchronous write and asynchronous read

```
module data_mem (
    input              CLK,
    input              WE,
    input       [31:0] A,
    input       [31:0] WD,
    output reg  [31:0] RD
);
    reg [7:0] memory [1023:0];

    always @(posedge CLK) begin
        if (WE) begin
            memory[A]     <= WD[7:0];
            memory[A + 1] <= WD[15:8];
            memory[A + 2] <= WD[23:16];
            memory[A + 3] <= WD[31:24];
        end
    end

    always @(*) begin
        RD[7:0]   = memory[A];
        RD[15:8]  = memory[A + 1];
        RD[23:16] = memory[A + 2];
        RD[31:24] = memory[A + 3];
    end
endmodule
```

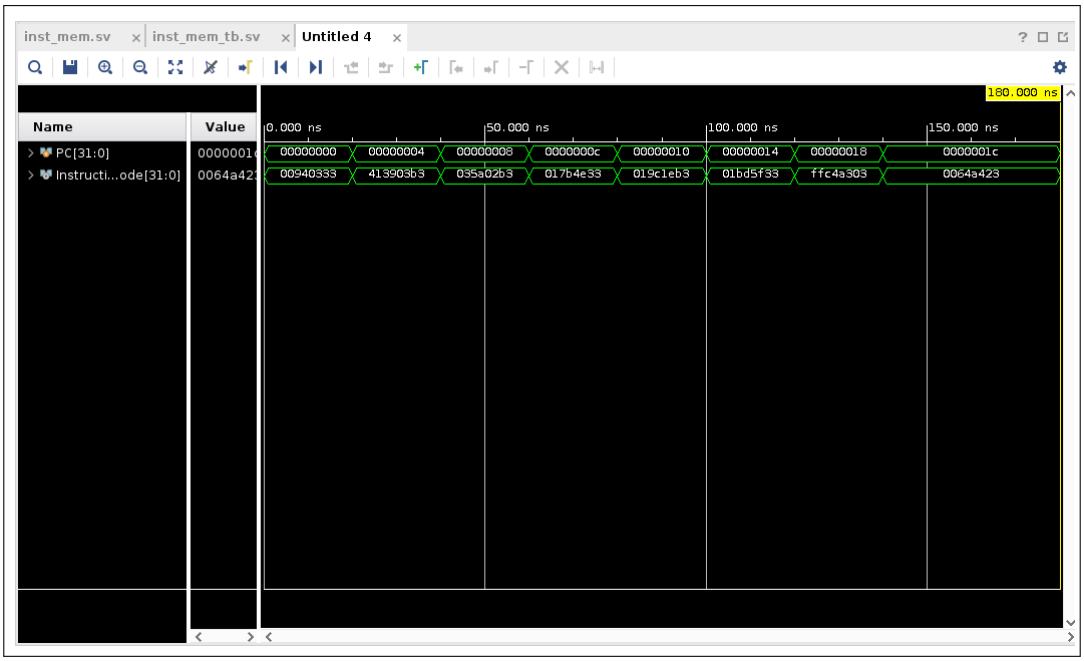

Figure 10: Instruction Memory output sequence based on input Program Counter.

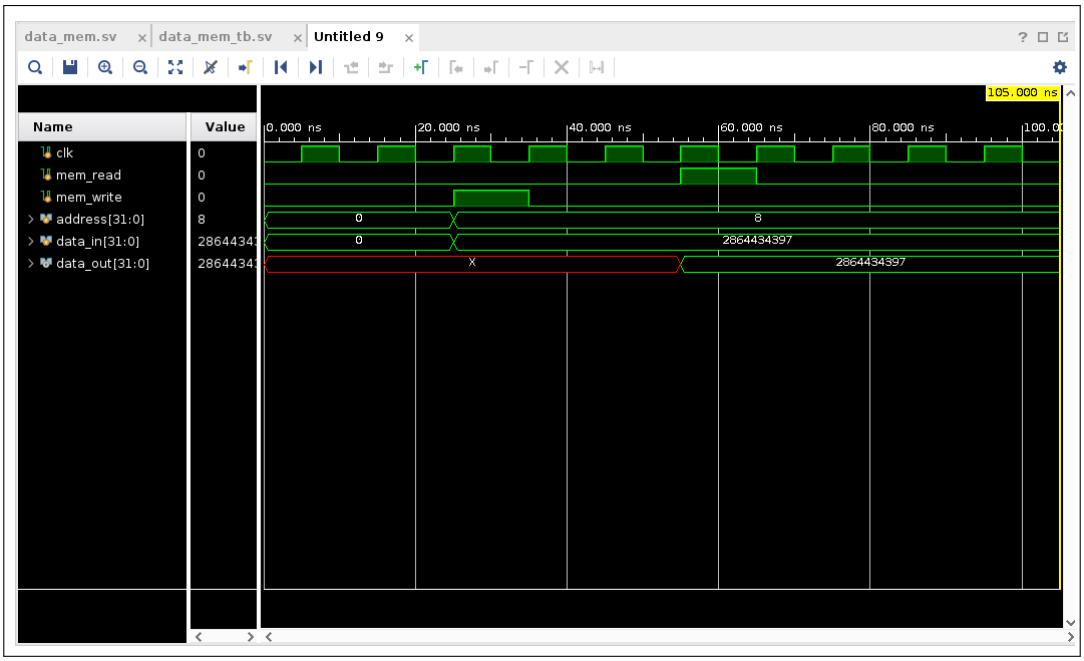

Figure 11: Data Memory verifying byte-addressable write and read operations.

5 Top-Level Integration and Processor Execution

The ultimate verification of the processor was performed by executing a complete program using the `top.sv` and `top_tb.sv` modules. The test program was hardcoded into the instruction memory as follows, inspired by typical RISC-V test examples [1, 5]:

Instruction Memory Initialization Sequence

```
// Address: Data
0x00: 00C001EF  // JAL x3, 12
0x04: 00108133  // ADD x2, x1, x1
0x08: 00202223  // SW x2, 4(x0)
0x0C: 00002083  // LW x1, 0(x0)
0x10: FE000AE3  // BEQ x0, x0, -12
```

This program tests the logic of every instruction type. Our testbench initialized `data_mem[0]` to the value 42. At address $0x0C$, the `LW` instruction successfully loaded 42 into register $x1$. Consequently, at $0x04$, the `ADD x2, x1, x1` instruction added $x1$ to itself and correctly stored 84 into $x2$. Finally, at $0x08$, the `SW` instruction successfully stored the value 84 into memory address 4. The complete testbench code can be found in Appendix A.

Top-level module instantiation (top.sv) showing component connections

```
module top (
    input clk, rst,
    output wire [31:0] pc,
    output wire [31:0] insn
);
    // internal wires ...
    mux_2x1 mux_1 (.out(mux_1_to_pc), .x(adder_1_to_mux_1), .y(
    adder_2_to_mux_1), .s(control_to_mux_1));
    PC_32bit pc_1 (.clk(clk), .rst(rst), .pc_in(mux_1_to_pc), .pc_out(
    pc_out));
    adder_32 add_1 (.a(pc_out), .b(32'd4), .sum(adder_1_to_mux_1));
    adder_32 add_2 (.a(pc_out), .b(extend_out), .sum(adder_2_to_mux_1));
    imm_Gen imm (.instruction(inst_mem_out), .immediate_output(extend_out
    ), .ImmSrc(control_to_extend));
    INST_MEM inst (.PC(pc_out), .Instruction_Code(inst_mem_out));
    // ... register file, ALU, data memory, control unit instantiation
    REG_FILE reg_f (
        .clock(clk), .reset(rst),
        .read_reg_num1(inst_mem_to_reg_file_1),
        .read_reg_num2(inst_mem_to_reg_file_2),
        .write_reg(inst_mem_to_reg_file_3),
        .write_data(mux_3_to_reg_file),
        .read_data1(read_data_1_wire),
        .read_data2(read_data_2_wire),
        .regwrite(reg_write_to_reg_file)
    );
    ALU alu (.in1(read_data_1_wire), .in2(mux_2_to_ALU), .result(
    ALU_result_wire), .zero_flag(zero_out), .alu_control(aluControl));
    data_mem data_m (.CLK(clk), .WE(mem_write_out), .A(ALU_result_wire),
    .WD(read_data_2_wire), .RD(read_data_from_mem_to_mux_3));
    control_unit control_dec (
        .op(Op), .RegWrite(reg_write_to_reg_file), .ImmSrc(
    control_to_extend),
        .ALUSrc(ALUSrc_to_mux_2), .MemWrite(mem_write_out), .ResultSrc(
    ResultSrc_to_mux3),
        .funct3(funct3), .funct7_5(funct7_5), .Zero(zero_out),
        .PCSrc(control_to_mux_1), .ALUControl(aluControl)
    );
    // ... remaining wire assignments
endmodule
```

These are the logs of the Top level simulation. Here we can see in detail how the operations are being performed, with all the relevant objects (wires and registers).

Simulation Execution Log

```
# KERNEL: Time: 0 | PC:   x | INSN: xxxxxxxx | ALU_Out:   x | RegW: x | x1: 00000000 | x2: 00000000 | x3: 00000000 | Mem[0]: 0000002a
# KERNEL: Time: 5000 | PC:   0 | INSN: 00c001ef | ALU_Out:   x | RegW: 1 | x1: 00000000 | x2: 00000000 | x3: 00000000 | Mem[0]: 0000002a
# KERNEL: Time: 15000 | PC:  12 | INSN: 00002083 | ALU_Out:   0 | RegW: 1 | x1: 00000000 | x2: 00000000 | x3: 00000004 | Mem[0]: 0000002a
# KERNEL: Time: 25000 | PC:  16 | INSN: fe000ae3 | ALU_Out:   0 | RegW: 0 | x1: 0000002a | x2: 00000000 | x3: 00000004 | Mem[0]: 0000002a
# KERNEL: Time: 35000 | PC:   4 | INSN: 00108133 | ALU_Out:  84 | RegW: 1 | x1: 0000002a | x2: 00000000 | x3: 00000004 | Mem[0]: 0000002a
# KERNEL: Time: 45000 | PC:   8 | INSN: 00202223 | ALU_Out:   4 | RegW: 0 | x1: 0000002a | x2: 00000054 | x3: 00000004 | Mem[0]: 0000002a
# KERNEL: Time: 55000 | PC:  12 | INSN: 00002083 | ALU_Out:   0 | RegW: 1 | x1: 0000002a | x2: 00000054 | x3: 00000004 | Mem[0]: 0000002a
# KERNEL: Time: 65000 | PC:  16 | INSN: fe000ae3 | ALU_Out:   0 | RegW: 0 | x1: 0000002a | x2: 00000054 | x3: 00000004 | Mem[0]: 0000002a
# KERNEL: Time: 75000 | PC:   4 | INSN: 00108133 | ALU_Out:  84 | RegW: 1 | x1: 0000002a | x2: 00000054 | x3: 00000004 | Mem[0]: 0000002a
# KERNEL: Time: 85000 | PC:   8 | INSN: 00202223 | ALU_Out:   4 | RegW: 0 | x1: 0000002a | x2: 00000054 | x3: 00000004 | Mem[0]: 0000002a
# KERNEL: Time: 95000 | PC:  12 | INSN: 00002083 | ALU_Out:   0 | RegW: 1 | x1: 0000002a | x2: 00000054 | x3: 00000004 | Mem[0]: 0000002a
# KERNEL: Time: 105000 | PC:  16 | INSN: fe000ae3 | ALU_Out:   0 | RegW: 0 | x1: 0000002a | x2: 00000054 | x3: 00000004 | Mem[0]: 0000002a
# KERNEL: Time: 115000 | PC:   4 | INSN: 00108133 | ALU_Out:  84 | RegW: 1 | x1: 0000002a | x2: 00000054 | x3: 00000004 | Mem[0]: 0000002a
# KERNEL: Time: 125000 | PC:   8 | INSN: 00202223 | ALU_Out:   4 | RegW: 0 | x1: 0000002a | x2: 00000054 | x3: 00000004 | Mem[0]: 0000002a
# KERNEL: Time: 135000 | PC:  12 | INSN: 00002083 | ALU_Out:   0 | RegW: 1 | x1: 0000002a | x2: 00000054 | x3: 00000004 | Mem[0]: 0000002a
# KERNEL: Time: 145000 | PC:  16 | INSN: fe000ae3 | ALU_Out:   0 | RegW: 0 | x1: 0000002a | x2: 00000054 | x3: 00000004 | Mem[0]: 0000002a
# KERNEL: Time: 155000 | PC:   4 | INSN: 00108133 | ALU_Out:  84 | RegW: 1 | x1: 0000002a | x2: 00000054 | x3: 00000004 | Mem[0]: 0000002a
# KERNEL: Time: 165000 | PC:   8 | INSN: 00202223 | ALU_Out:   4 | RegW: 0 | x1: 0000002a | x2: 00000054 | x3: 00000004 | Mem[0]: 0000002a
# KERNEL: Time: 175000 | PC:  12 | INSN: 00002083 | ALU_Out:   0 | RegW: 1 | x1: 0000002a | x2: 00000054 | x3: 00000004 | Mem[0]: 0000002a
# KERNEL: Time: 185000 | PC:  16 | INSN: fe000ae3 | ALU_Out:   0 | RegW: 0 | x1: 0000002a | x2: 00000054 | x3: 00000004 | Mem[0]: 0000002a
# KERNEL: Time: 195000 | PC:   4 | INSN: 00108133 | ALU_Out:  84 | RegW: 1 | x1: 0000002a | x2: 00000054 | x3: 00000004 | Mem[0]: 0000002a
# KERNEL: Time: 205000 | PC:   8 | INSN: 00202223 | ALU_Out:   4 | RegW: 0 | x1: 0000002a | x2: 00000054 | x3: 00000004 | Mem[0]: 0000002a
# RUNTIME: Info: RUNTIME_0068 testbench.sv (51): $finish called.
# KERNEL: Time: 212 ns,  Iteration: 0,  Instance: /tb_top,  Process: @INITIAL#18_2@.
# KERNEL: stopped at time: 212 ns
```

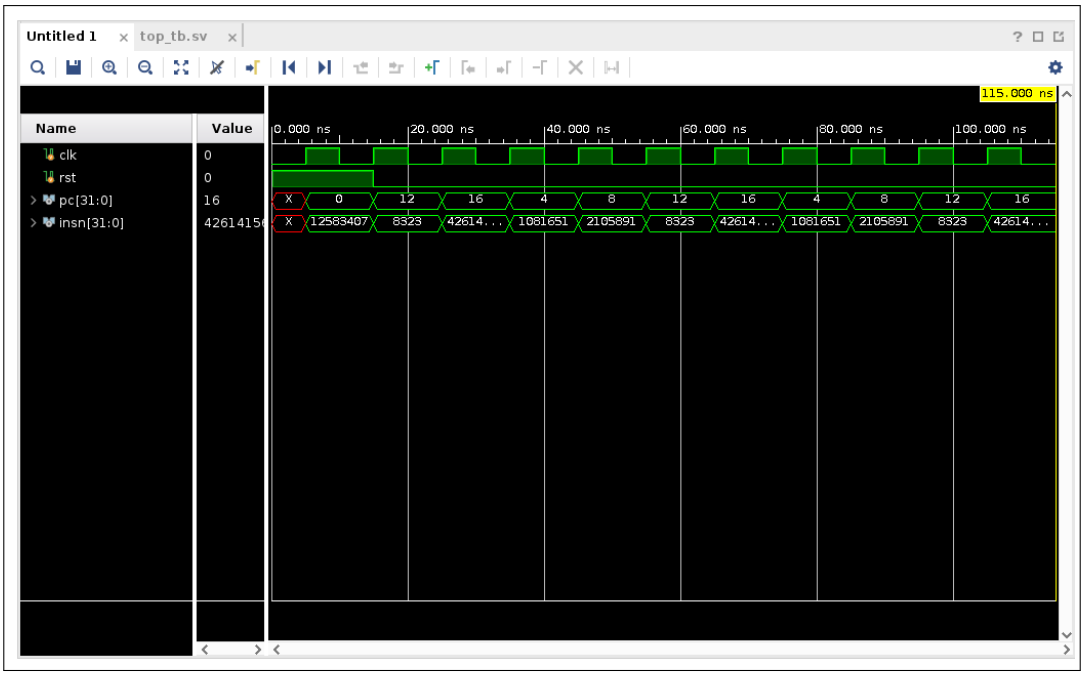

Figure 12: Top-level integration simulation showing the PC sequence and instruction fetch.

The Program Counter (`pc`) shows the expected sequence:

$$0 \rightarrow 12 \rightarrow 16 \rightarrow 4 \rightarrow 8 \rightarrow 12 \rightarrow 16 \rightarrow 4 \ldots$$

This sequence matches our expected behavioral flow: the JAL jumps to 0x0C, the LW executes, the BEQ jumps back to 0x04, and the ADD/SW execute before looping again. This proves the efficacy of the complete hierarchical control logic.

It is also worth noting that the top-level simulation exhibits an initial X (unknown) state on the `insn` output prior to the assertion of the reset signal. This is expected behavior in simulation as the instruction memory and PC are uninitialized. Upon `rst` going high, the processor correctly synchronizes and begins executing from address $0x00$.

6 Conclusion

Summary

Through rigorous unit testing and a successful integration test, the Single-Cycle RISC-V processor has been verified to functionally support R, I, S, B, and J type instructions. All modules behaved according to architectural specifications, and the PC trace confirms accurate execution of branching and jumping operations. The design matches the expected behavior as described in the RISC-V specification [1, 7] and standard single-cycle microarchitecture [2, 6]. The verification methodology, employing directed testbenches for each module and a comprehensive top-level test, ensures a high level of confidence in the design's correctness [4, 5, 8, 9].

References

[1] A. Waterman and K. Asanovic, *The RISC-V Instruction Set Manual, Volume I: User-Level ISA*, Document Version 2.2, RISC-V Foundation, May 2017.

[2] J. L. Hennessy and D. A. Patterson, *Computer Organization and Design: The Hardware/-Software Interface, RISC-V Edition*, Morgan Kaufmann, 2017.

[3] IEEE Standard for Verilog Hardware Description Language, IEEE Std 1364-2005, 2006.

[4] C. Spear and G. Tumbush, *SystemVerilog for Verification: A Guide to Learning the Test-bench Language Features*, 2nd ed., Springer, 2008.

[5] D. M. Harris and S. L. Harris, *Digital Design and Computer Architecture: RISC-V Edition*, Morgan Kaufmann, 2022.

[6] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed., Morgan Kaufmann, 2017.

[7] A. Waterman, Y. Lee, D. Patterson, and K. Asanovic, *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture*, Document Version 1.11, RISC-V Foundation, 2017.

[8] F. Vahid, *Digital Design with RTL Design, VHDL, and Verilog*, 2nd ed., Wiley, 2011.

[9] IEEE Standard for SystemVerilog—Unified Hardware Design, Specification, and Verification Language, IEEE Std 1800-2017, 2018.

[10] D. Patterson and A. Waterman, *The RISC-V Reader: An Open Architecture Atlas*, Strawberry Canyon, 2017.

A Appendix A: Top-Level Testbench (tb_top.sv)

The following code is the complete top-level testbench used for the final integration simulation. It initializes the processor, pre-loads data memory, and monitors key internal signals.

Top-Level Testbench (tb_top.sv)

```
`timescale 1ns / 1ps

module tb_top;

    reg clk;
    reg rst;

    wire [31:0] pc;
    wire [31:0] insn;

    top uut (.clk(clk), .rst(rst), .pc(pc), .insn(insn));

    // Clock Generation (10ns period)
    always begin
        #5 clk = ~clk;
    end

    initial begin
        // 1. Initialize
        clk = 0;
        rst = 1;

        // 2. Pre-load Data Memory
        uut.data_m.memory[0] = 8'h2A; // LSB (Decimal 42)
        uut.data_m.memory[1] = 8'h00;
        uut.data_m.memory[2] = 8'h00;
        uut.data_m.memory[3] = 8'h00; // MSB

        $monitor("Time: %0t | PC: %3d | INSN: %h | ALU_Out: %3d | RegW: %b | x1: %h | x2: %h | x3: %h | Mem[0]: %h",
                 $time,
                 pc,
                 insn,
                 uut.ALU_result_wire,
                 uut.reg_write_to_reg_file,
                 uut.reg_f.reg_memory[1],  // Peek at register x1
                 uut.reg_f.reg_memory[2],
                 uut.reg_f.reg_memory[3],// Peek at register x2
                 {uut.data_m.memory[3], uut.data_m.memory[2], uut.data_m.memory[1], uut.data_m.memory[0]}
        );


        #12;
        rst = 0;
        #200;

        $finish;
    end

endmodule
```

B Appendix B: Top Module (top.sv)

This appendix contains the full top-level module code, which instantiates all datapath components and connects the control unit.

Top Module (top.sv)

```
`timescale 1ns / 1ps

module top (
    input clk, rst,
    output wire [31:0] pc,
    output wire [31:0] insn
);

    wire [31:0] mux_1_to_pc;
    wire [31:0] pc_out;
    wire [31:0] adder_1_to_mux_1;
    wire [31:0] adder_2_to_mux_1;
    wire [31:0] extend_out;
    wire [31:0] inst_mem_out;
    wire [31:0] read_data_1_wire;
    wire [31:0] read_data_2_wire;
    wire [31:0] mux_2_to_ALU;
    wire [31:0] ALU_result_wire;
    wire        zero_out;
    wire [31:0] read_data_from_mem_to_mux_3;
    wire        mem_write_out;
    wire [31:0] mux_3_to_reg_file;

    wire [4:0]  inst_mem_to_reg_file_1;
    wire [4:0]  inst_mem_to_reg_file_2;
    wire [4:0]  inst_mem_to_reg_file_3;

    wire [6:0]  Op;
    wire        reg_write_to_reg_file;
    wire [1:0]  control_to_extend;
    wire        ALUSrc_to_mux_2;
    wire [1:0]  ResultSrc_to_mux3;
    wire        control_to_mux_1;
    wire [2:0]  aluControl;

    // Instantiations
    mux_2x1 mux_1 (.out(mux_1_to_pc), .x(adder_1_to_mux_1), .y(
    adder_2_to_mux_1), .s(control_to_mux_1));

    PC_32bit pc_1 (.clk(clk), .rst(rst), .pc_in(mux_1_to_pc), .pc_out(
    pc_out));

    adder_32 add_1 (.a(pc_out), .b(32'd4), .sum(adder_1_to_mux_1));

    adder_32 add_2 (.a(pc_out), .b(extend_out), .sum(adder_2_to_mux_1));

    imm_Gen imm (.instruction(inst_mem_out), .immediate_output(extend_out
    ), .ImmSrc(control_to_extend));

    INST_MEM inst (.PC(pc_out), .Instruction_Code(inst_mem_out));

    assign inst_mem_to_reg_file_1 = inst_mem_out[19:15];
    assign inst_mem_to_reg_file_2 = inst_mem_out[24:20];
    assign inst_mem_to_reg_file_3 = inst_mem_out[11:7];
```

```
52      assign Op = inst_mem_out[6:0];
53
54      REG_FILE reg_f (
55          .clock(clk),
56          .reset(rst),
57          .read_reg_num1(inst_mem_to_reg_file_1),
58          .read_reg_num2(inst_mem_to_reg_file_2),
59          .write_reg(inst_mem_to_reg_file_3),
60          .write_data(mux_3_to_reg_file),
61          .read_data1(read_data_1_wire),
62          .read_data2(read_data_2_wire),
63          .regwrite(reg_write_to_reg_file)
64      );
65
66      mux_2x1 mux_2 (.out(mux_2_to_ALU), .x(read_data_2_wire), .y(
        extend_out), .s(ALUSrc_to_mux_2));
67
68      ALU alu (.in1(read_data_1_wire), .in2(mux_2_to_ALU), .result(
        ALU_result_wire), .zero_flag(zero_out), .alu_control(aluControl));
69
70      data_mem data_m (.CLK(clk), .WE(mem_write_out), .A(ALU_result_wire),
        .WD(read_data_2_wire), .RD(read_data_from_mem_to_mux_3));
71
72
73      mux_3x1 mux_3 (
74          .out(mux_3_to_reg_file),
75          .d0(ALU_result_wire),
76          .d1(read_data_from_mem_to_mux_3),
77          .d2(adder_1_to_mux_1),
78          .s(ResultSrc_to_mux3)
79      );
80
81      wire funct7_5;
82      wire [2:0] funct3;
83      assign funct3   = inst_mem_out[14:12];
84      assign funct7_5 = inst_mem_out[30];
85
86      assign pc   = pc_out;
87      assign insn = inst_mem_out;
88
89      control_unit control_dec (
90          .op(Op),
91          .RegWrite(reg_write_to_reg_file),
92          .ImmSrc(control_to_extend),
93          .ALUSrc(ALUSrc_to_mux_2),
94          .MemWrite(mem_write_out),
95          .ResultSrc(ResultSrc_to_mux3),
96          .funct3(funct3),
97          .funct7_5(funct7_5),
98          .Zero(zero_out),
99          .PCSrc(control_to_mux_1),
100         .ALUControl(aluControl)
101     );
102
103 endmodule
```